

.NET

❖ What is .NET?

.NET is a software development framework & platform created by Microsoft => for building different types of applications.

Using .NET, developers can create:

- Web applications
- Desktop applications
- Mobile applications
- APIs & Games etc..

The most commonly used programming language with .NET is C#(C-Sharp).

❖ Why Do we use .NET?

.NET helps developers to write code faster, reuse existing libraries, build secure applications, support multiple platforms.

❖ Where can we use .NET?

1. Web Applications:

Using ASP.NET Core, developers create websites.

Ex:

- E-commerce websites
- College Management Systems / Employee Management Systems.

Amazon-like website

↓

Frontend: HTML, CSS, Javascript

Backend: ASP.NET Core

Database: SQL Server

2. APIs:

API allows applications to communicate

Ex: A mobile app requests for student details.

Mobile APP

↓ API Request

ASP.NET Core API

↓

SQL Database

3. Desktop applications:

It is used for windows forms and Windows Presentation Foundation (WPF)

Ex:

- Calculator
- Hospital Management System
- Billing Software

❖ How .NET Works?

```
Console.WriteLine("HelloWorld");
```

Step-1: C# Code

Class Program

```
{  
    static void Main()  
    {  
        Console.WriteLine("Hello");  
    }  
}
```

Step-2: Compilation

The C# compiler converts code into Intermediate Language (IL).

IL is platform-Independent.

C# Code → IL Code

Step-3: CLR Execution

CLR converts Intermediate Language into machine code using JIT (Just in Time Compiler).

IL → Machine Code

❖ Architecture of .NET

Application → C# Code → Compiler → IL → CLR → Operating System

❖ Important Components of .NET

1. CLR:

- CLR stands for Common Language Runtime.
- CLR is the heart of .Net
- It manages Memory, Exception handling, Security, Garbage collection, Thread.

Ex: int x = 10;

Memory allocation is handled automatically by CLR.

2. CTS:

- CTS stands for Common Type System.
- It is used to define data types.

Ex:

```
int age = 20
```

```
string name = "John";
```

All .Net languages follow CTS.

3. CLS:

- CLS stands for Common Language Specification.
- Rules that all .Net languages follow.

Ex:

A library written in C# can be used in VB.NET because of Common Language Specification.

4. FCL:

- FCL stands for Framework Class Library.
- It is a collection of pre-written classes.

Ex:

```
Console.WriteLine();
```

```
Math.Sqrt();
```

```
List<int>();
```

```
StringBuilder();
```

Instead of writing everything from scratch, developers use these libraries.

5. GC:

- GC stands for Garbage Collector.
- It automatically frees unused memory.
- It removes unused objects from memory.

Ex:

```
Student s = new Student();
```

```
s = null;
```

❖ Concepts of .NET

1. Variables:

Variables are used for storing data.

Ex:

```
int age = 19;
```

```
string name = "Raju";
```

2. Data Types:

Data Types are classified into two parts:

1. **Value Types:** It will stores only actual values.

Ex:

int, float, double, char, bool

2. **Reference Types:** In this process the data is stored by reference.

Ex:

class, object, string, array.

3. **Operators:** It is used to perform operations using operand. Most commonly used operators are:

+, -, *, /, ==, !=, &&, ||

Ex:

```
int a = 6;
```

```
int b = 2;
```

```
Console.WriteLine(a+b);
```

O/p: 8

4. **Conditional Statements:** It controls the flow of execution by evaluating expressions as true or false.

- **if:-**

```
if(age>=18)
```

```
{
```

```
    Console.WriteLine("Eligible");
```

```
}
```

- **if-else:-**

```
if(age>=18)
```

```
    Console.WriteLine("Adult");
```

```
else
```

```
    Console.WriteLine("Minor");
```

5. **Loops:** Loops allow us to execute a block of code repeatedly based on specific conditions.

- **for loop:-**

```
for(int i = 1; i <= 5; i++)
```

```
{
```

```
    Console.WriteLine(i);
```

```
}
```

O/p: 1 2 3 4 5

- **while loop:-**

```
while(i < 5)

{

    i++;

}
```

- **foreach loop:-**

```
int[] nums = { 1, 2, 3};

foreach(int n in nums)

{

    Console.WriteLine(n);

}
```

6. Arrays: An array is a collection of elements of the same type stored in contiguous memory locations

Ex:

```
int[] marks={90, 80, 70};
```

7. Method: Method is a reusable block of code that performs a specific action.

Ex:

```
int Add(int a, int b)

{

    return a+b;

}
```

call:- Add(2,3);

O/p: 5

8. Classes & Objects:

- **Class:**

Class is a blueprint.

Ex:

```
class Student

{

    public string Name;

}
```

- **Object:**

Object is an instance of class.

Ex:

```
Student s = new Student();  
s.Name = "Nishita";  
Console.WriteLine(s.Name);
```

O/p: Nishita

9. Constructors: Constructor is automatically called when an object is created.

Ex:

```
class Student  
{  
    public Student()  
    {  
        Console.WriteLine("Created");  
    }  
}
```

10. Encapsulation: It is the process of wrapping code and data together into a single unit, like a class.

Ex:

```
private int age;  
public void SetAge(int a)  
{  
    Age = a;  
}
```

11. Inheritance: It allows a new class to acquire the properties of an existing class.

Ex:

```
class Animal  
{  
    public void Eat()  
    {  
        Console.WriteLine("Eating");  
    }  
}  
class Dog : Animal  
{  
}
```

12. Polymorphism: It allows a single method or object to behave differently in different contexts. It has 2 types:

- **Method Overloading:**

```
Add(int a, int b)
```

```
Add(int a, int b, int c)
```

- **Method Overriding:**

```
public virtual void Sound() {}
```

```
public override void Sound() {}
```

13. Abstraction: It is about hiding the complex implementation details and showing only essential features to the user.

Ex:

```
abstract class Shape
```

```
{
```

```
    public abstract void Draw();
```

```
}
```

14. Interface: An interface is a contract. It defines what a class must do, but not how to do it.

Ex-1:

```
interface IAnimal
```

```
{
```

```
    void Speak();
```

```
}
```

Ex-2:

```
class Dog : IAnimal
```

```
{
```

```
    public void Speak()
```

```
    {
```

```
        Console.WriteLine("Bark");
```

```
    }
```

```
}
```

15. Collections: Collections are structures used to store and manage groups of related objects.

- **List:**

```
List<string> names=new List<string>();
```

- **Dictionary:**

```
Dictionary<int, string> students=new Dictionary<int, string>();
```